

RW-TTT: Batched Serving for Request-Owned Test-Time Training State

Jian Yang¹ Zhizhuo Kou¹ Yao Tian¹ Hao Zhang² Han Chen³ Sirui Han¹ Yike Guo¹
¹HKUST ²CUHK ³NUS

jyang827@connect.ust.hk, zkouaa@connect.ust.hk, ytianbc@cse.ust.hk
 zhanghaowuda12@gmail.com, chenhan@u.nus.edu, siruihan@ust.hk, yikeguo@ust.hk

Abstract

Test-time training (TTT) adapts an LLM during generation by reading and updating request-owned state, such as fast weights, low-rank deltas, or streaming learner state. This breaks batched LLM serving, which assumes shared static weights: serial execution is correct but slow, while naive batching can corrupt request state. We formulate this problem as *read-write TTT serving* and present RW-TTT, which tags each decode step with its owner, version, and READ/WRITE effect, batches only compatible phases, and commits updates only to the owner. On one GPU with eight fast-weight In-Place-TTT streams, RW-TTT reaches 274.61 aggregate tok/s, $9.31\times$ over sequential serving and $3.44\times$ over per-stream replicas under the same memory budget. It preserves behavior on RULER, a long-context benchmark, and passes owner/version checks.

1 Introduction

Modern LLM serving batches requests over shared, read-only weights. vLLM manages dynamic activation-side state—paged KV cache—but every request still reads the same model parameters (Yu et al., 2022; Kwon et al., 2023; Agrawal et al., 2023, 2024; Zhong et al., 2024; Zheng et al., 2024). Adapter-serving systems such as S-LoRA and Punica add request-specific *model*-side state, yet those adapters are pre-existing and read-only during generation (Sheng et al., 2023; Chen et al., 2024). Test-time training (TTT) breaks this contract (Sun et al., 2020, 2024; Feng et al., 2026). A TTT request may carry mutable model-side adaptation state that is both read and *updated* during decoding. In In-Place TTT, this state is a set of request-specific fast weights in the MLP down-projection (Feng et al., 2026). In LoRA-style test-time learning, the mutable state can be represented as low-rank parameter deltas rather than full-model updates (Hu et al., 2025). In end-to-end TTT for

long context, the model continues learning from the incoming context, yielding streaming learner state that changes as the context is processed (Tan-don et al., 2025). This state is request-owned: it must not leak to other requests, and its updates must follow the backend’s prescribed state transition. A serial TTT path can keep this transition implicit by processing one stream at a time, but batched serving must make it explicit. In particular, if a write fails, falls back to a sequential path, or is rolled back after speculative execution, the runtime must know which state version remains valid. We call the resulting tension *read-write TTT serving*: READ steps apply request-owned state without mutation, while WRITE steps update the state and commit a new version.

We propose RW-TTT, a serving system for this setting. RW-TTT tags each serving step with its owner and READ/WRITE effect, groups only compatible phases into legal batches, executes read and write operators separately, and commits updates exclusively to the owning requests. To support rollback, failed-write recovery, fallback paths, and speculative branches, RW-TTT maintains explicit state versions: READ steps preserve version v , while successful WRITE steps commit version $v + 1$. The same contract covers the mutable-state forms above—fast weights, low-rank deltas, and streaming learner state. The backend supplies the payload and update rule; RW-TTT supplies ownership, versioning, rollback-safe commit, and batch planning. On a long-context serving workload with a fast-weight In-Place-TTT checkpoint, RW-TTT reaches 274.61 aggregate tok/s, a $9.31\times$ gain over the sequential TTT path and a $3.44\times$ gain over naive independent replicas. RULER (Hsieh et al., 2024) long-context evaluations show that RW-TTT preserves the adapted checkpoint’s task behavior.

The paper makes three contributions.

1. **A read-write TTT serving contract.** We formulate the serving problem for online-adaptive TTT checkpoints and define its core primitives: request ownership, state versions, READ/WRITE effects, and owner-preserving commit (§3).
2. **An owner/version-aware mutable-state batching planner.** We design a planner that separates READ and WRITE phases, groups only compatible next steps, and commits updated versions exclusively to correct request owners (§4).
3. **End-to-end serving evidence with behavior and correctness preservation.** On the evaluated fast-weight TTT checkpoint, RW-TTT recovers $9.31 \times$ aggregate throughput over the sequential path while preserving RULER long-context evaluation (32K/64K) scores and passing owner/version correctness gates. (§5).

2 Background

2.1 Serving with Mutable Model State

Autoregressive serving has two phases: *prefill*, which builds the KV cache from the prompt, and *decode*, which generates tokens while extending that cache. Existing serving systems largely rely on an assumption of immutable model-side state.

- **Static-weight serving.** Model parameters are shared across requests and are not modified during generation. This assumption underlies continuous batching in OrCA (Yu et al., 2022), paged KV-cache management in vLLM (Kwon et al., 2023), and structured-generation scheduling in SGLang (Zheng et al., 2024).
- **Adapter serving.** S-LoRA (Sheng et al., 2023) and Punica (Chen et al., 2024) batch requests with different adapters (Hu et al., 2022; Mangrulkar et al., 2022), but each adapter is fixed during generation.

Test-time training (TTT) invalidates this assumption. TTT methods adapt model computation using inference-time information (Sun et al., 2020, 2024; Feng et al., 2026). From a serving perspective, the defining property is *request-owned mutable model-side state*: each request may carry fast weights, low-rank deltas, or tail buffers that evolve with its

local context. Serving such requests requires two logical operations:

- **READ**, which applies the request’s current state version during token computation; and
- **WRITE**, which updates the owning request’s state, typically at chunk boundaries such as every C_{ttt} generated tokens.

Because requests arrive independently and advance at different rates, one request may be updating its state while others are still reading earlier state versions. Thus, READ and WRITE operations are not globally synchronized across requests.

2.2 Serving Gap

Table 1 summarizes the gap. Existing runtimes manage either request-local activation state, as in KV-cache systems, or immutable request-specific model-side state, as in adapter-serving systems. TTT instead requires *owned*, *versioned*, *mutable* model-side state. Compatible READs can be batched because they only observe existing versions. WRITES, however, mutate the owning request’s state and must not become visible to incompatible READs. A TTT serving runtime must therefore track whose state a WRITE updates, which state version each READ observes, and when an updated version becomes visible.

A straightforward way to support TTT is to serialize or isolate requests so that READ and WRITE operations cannot interfere. As shown in Table 2, these baselines preserve correctness but leave substantial batching opportunities unused. Sequential serving reaches only 29.51 tok/s with eight streams. Independent replicas improve throughput to 79.78 tok/s using three batch-1 processes under ~ 33 GiB, but still treat requests as isolated executions and therefore cannot share compute across compatible READ phases.

This leaves the central runtime question: *how can a serving system batch requests with mutable model-side state while preserving ownership and version semantics?*

3 Serving Model

Read-write TTT serving is the problem of batching generation requests whose model-side adaptation state is request-owned and mutable. Given active requests $\mathcal{R}_t$, the serving system must decide which next transitions to issue together, and when to issue them, while preserving each request’s owner,

System / abstraction	Native request state	Online WRITE	Versioned commit	Native batching key
vLLM / PagedAttention	KV cache	–	–	token position / KV page
SGLang	KV / prefix cache	–	–	prefix or program state
Punica / S-LoRA	pre-existing adapter	–	–	adapter id / rank
Sequential TTT path	mutable fast state	✓	implicit / serial	single stream
RW-TTT	request-owned TTTState	✓	✓	R/W phase + version + state

Table 1: Serving-contract comparison. Existing systems batch read-only request state: KV/prefix caches or pre-existing adapters. Read-write TTT instead batches request-owned model state that mutates during decoding, requiring explicit ownership, versioned commit, and READ/WRITE-aware batching.

Execution strategy	Streams/processes	Throughput
Sequential serving	8 streams	29.51 tok/s
Independent replicas	3 batch-1 processes	79.78 tok/s

Table 2: Throughput of straightforward TTT serving baselines on the evaluated In-Place-TTT checkpoint (Feng et al., 2026). Sequential execution preserves READ/WRITE semantics but loses batching opportunities, while independent replicas improve throughput by isolation but cannot share compute across compatible READ phases.

state version, read/write effect, and sequential TTT behavior. The goal is to recover legal batching capacity under the semantic constraints introduced by online mutation.

A request r contains ordinary inference state and mutable TTT state:

$$q_r = (x_r, k_r, p_r, s_r^v, m_r), \quad (1)$$

where x_r is the current token or hidden input, k_r is KV/cache state, p_r is decode position, s_r^v is the versioned request-owned TTT state, and m_r stores serving-visible metadata such as tail length and update readiness. Base model parameters remain shared across requests; the mutable state s_r^v does not.

Ownership. Each mutable TTT state has exactly one owner. A read transition may consume the owner’s current state without changing its version:

$$\text{READ}_\theta(r, x_r, s_r^v) \rightarrow (y_r, s_r^v).$$

A write transition consumes backend update evidence u_r , produces a new state version, and must commit that version back to the same owner:

$$\text{WRITE}_\theta(r, x_r, u_r, s_r^v) \rightarrow (y_r, s_r^{v+1}).$$

This ownership rule distinguishes TTT state from shared model weights and from activation-only KV cache. A serving schedule may reorder independent owners for batching, but it may not redirect, merge, or alias their mutable states.

Legal batching. Read and write transitions are never mixed in the same operator group. More generally, a group is legal only when its requests share the same state effect, backend state type, and operator shape class, and when each request is at the expected committed version. The backend state type separates fast-weight TTT, LoRA-style delta TTT, and streaming learner state. The operator shape class records the kernel-relevant dimensions for that backend, such as dtype, layer set, hidden dimensions, chunk size, rank, or active-row count. These keys make the batching rule explicit: ordinary activation compatibility is not enough when requests also read and write private model-side state.

Formally, let the next transition exposed by request r_i be

$$e_i = (r_i, \tau_i, \sigma_i, \rho_i, v_i), \quad (2)$$

where τ_i is backend state type, σ_i is operator shape class, $\rho_i \in \{\text{READ}, \text{WRITE}\}$ is the state effect, and v_i is the expected state version. At decode step t , the planner observes candidate transitions $\mathcal{E}_t = \{e_i\}$ and emits legal groups $\mathcal{G}_t = \{G_k\}$. A group G is legal only if

$$\forall i, j \in G : (\tau_i, \sigma_i, \rho_i) = (\tau_j, \sigma_j, \rho_j), \quad (3)$$

$$v_i = V(r_i),$$

where $V(r_i)$ is the serving system’s current committed version for owner r_i . The group also carries an injective owner map: no two batch elements may commit to the same request-owned state slot in the same transition. Thus a legal read-write TTT batch is not merely an activation batch; it is a group of state-effect-compatible transitions with valid owner-version bindings.

Bounded waiting. The planner may delay a ready transition to improve compatibility, but only within a bounded wait budget. If transition e for

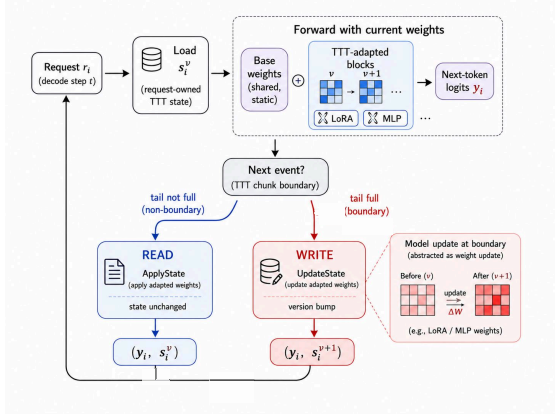


Figure 1: Per-request READ/WRITE semantics. Each decode step evaluates the shared base model with the request-owned TTT state and exposes either a version-preserving READ event or a version-creating WRITE event.

request r becomes ready at decode step $t_{\text{ready}}(r, e)$ and is issued at $t_{\text{issue}}(r, e)$, then

$$0 \leq t_{\text{issue}}(r, e) - t_{\text{ready}}(r, e) \leq w. \quad (4)$$

Thus $w = 0$ is greedy batching, and finite w bounds starvation in decode steps while permitting short phase-alignment waits. Waiting may change which compatible requests are issued together; it may not change an individual request’s read/write order or update rule.

Correctness contract. The target behavior is the original sequential TTT execution using the same backend and update rule. An execution schedule is behavior-preserving if it preserves request-output mapping, read-step immutability, write order and count, version progression, owner-local commit, and tail/cache consistency for every request. In experiments, we validate this semantic contract through owner/version/tail invariants and by checking that per-token NLL and task scores match the sequential TTT reference within the reported tolerance. This is the serving contract: RW-TTT changes the execution plan, not the adapted TTT semantics.

4 RW-TTT Serving System

4.1 Serving a Mutable TTT Step

Figure 1 shows the per-request transition that the runtime must preserve. At each decode step, request r reads its owner-visible TTT state s_r^v and evaluates the shared base model with that state. A READ applies s_r^v and leaves version v visible to the

Primitive	Ph.	State	Contract
APPLYSTATE	R	weights / deltas	apply; keep version
TAILBUFFERUPDATE	R	tail metadata	append; no version bump
BOUNDARYUPDATE	W	TTT payload	compute dirty state
COMMIT/ROLLBACK	C	owner/version log	publish; restore; fork

Table 3: Core read-write TTT serving primitives. R/W/C denote read, write, and control phases. The backend supplies the payload and update rule; the serving runtime preserves owner/version semantics.

next step. A WRITE consumes backend-specific update evidence u_r and produces a dirty candidate state $\tilde{s}_r^{v+1}$, which becomes visible only after commit. Legal serving must therefore preserve output-to-request mapping, per-request READ/WRITE order, and owner-local version transitions (Section 3).

Figure 2 places this contract in the serving stack. Requests carry ordinary KV/cache state plus TTTState metadata: owner id, backend type, version, placement, and backend metadata. Payloads such as FastWeightState, DeltaAdapterState, and TailBufferState represent fast weights, adapter deltas, and update evidence. A StateView exposes the executable slice for the next step: read views are immutable, while write views carry update inputs and mark the candidate state dirty until commit.

The planner only observes each request’s next transition. It batches compatible effects and backend shapes automatically, checks that every owner is at its expected committed version, and uses the owner map μ to connect physical batch slots back to request semantics. For group $G = \{r_b\}_{b=1}^{B_G}$ with $\mu(b) = r_b$, READ outputs keep all versions unchanged, while WRITE outputs commit only the owners in $\mu(G)$. Selective commit and rollback use the same owner-indexed state table by snapshotting $c_r^v \leftarrow s_r^v$ before speculative writes.

4.2 Primitives

Table 3 lists the serving-visible primitive groups. A TTT backend owns the update rule; RW-TTT fixes how state is applied, when a candidate version is produced, and how successful writes become owner-visible.

READ. The backend decides how s_r^v affects model computation: fast weights, low-rank deltas, or another learner state. The runtime requires only that READ views are immutable and returned with the same version. Tail-buffer maintenance belongs

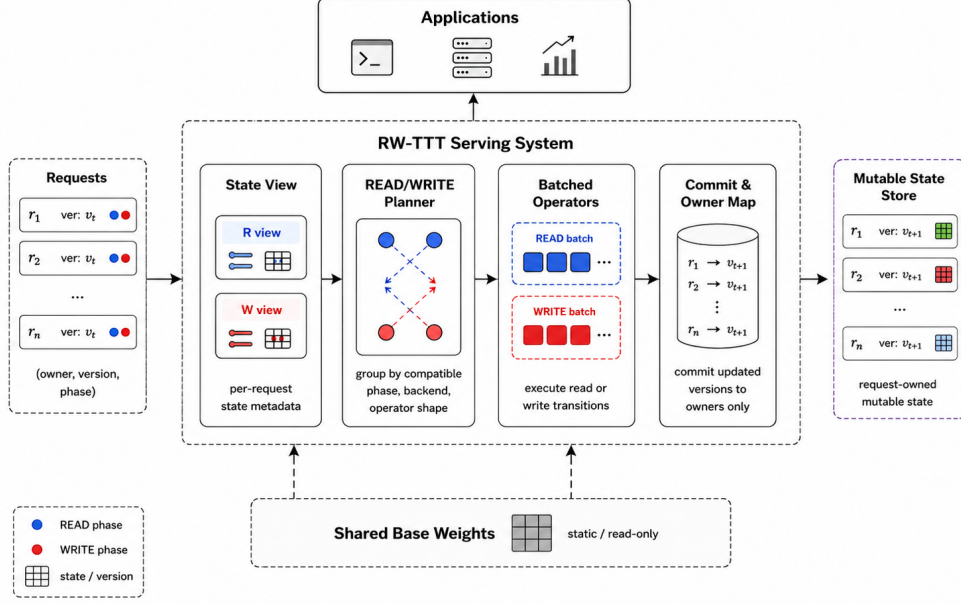


Figure 2: RW-TTT serving architecture. Requests carry owner-versioned mutable state alongside ordinary KV cache. The planner automatically groups compatible READ/WRITE transitions by backend type, operator shape, placement, and valid owner version; batched operators return outputs and updated state through the owner map. The figure shows serving semantics, not the backend update rule.

here because it records request-local evidence for a later update boundary without publishing a new owner-visible TTT state version.

WRITE. The update evidence u_r is backend-specific. RW-TTT enforces that the update reads owner r 's committed version v and produces one dirty candidate state for that owner. The candidate is invisible to later READs until committed. The Triton experiments in Section 5 target WRITE-side update and selective-commit kernels because ordinary static-weight kernels do not cover this state-management path.

CONTROL. COMMIT/ROLLBACK controls visibility. Commit publishes a dirty state as owner r 's next version only after the WRITE group succeeds; rollback restores the checkpointed owner slot and version; fork creates a separate owner/version lineage. The version counter changes here, not inside the backend update, and only owner-indexed TTT records are modified.

4.3 TTT-Aware Batch Planning

For each ready transition e_i , the planner computes $\kappa_i = (\rho_i, \tau_i, \sigma_i, \pi_i)$, covering effect, backend type, operator shape, and placement. The owner-local version v_i is checked against the committed ver-

sion $V(r_i)$, but different owners need not have equal numeric versions. Requests with different keys are never co-issued, so READ and WRITE steps are separated by construction. For each key, the planner emits a group at target batch size B or waits at most w decode steps for compatible arrivals. When the wait budget expires, it issues the current legal prefix after owner-version and owner-map checks. This changes only which independent owners are co-issued, not any request's local order or backend update rule.

Serving loop. Algorithm 1 summarizes the per-iteration control flow.

4.4 Backend Instantiations

This interface lets multiple TTT styles share the same serving contract. A backend exposes a request-owned payload, immutable read view, write evidence for its update rule, compatibility metadata (backend type, operator shape, and placement), and commit metadata for the owner slot and candidate version. The runtime uses only this interface; the backend retains its learning formula.

For example, a fast-weight backend stores mutable fast weights in FastWeightState; a LoRA-style backend stores request-owned low-rank deltas in DeltaAdapterState. Both expose READ ap-

Algorithm 1 RW-TTT TTT-aware serving loop**Require:** active requests $\mathcal{R}$, target batch size B , wait budget

```

 $w$ 
1: while  $\mathcal{R}$  is not empty do
2:    $\mathcal{E} \leftarrow \emptyset$ 
3:   for all  $r \in \mathcal{R}$  do
4:      $v_r \leftarrow \text{VIEW}(r.\text{TTTState}, r.\text{metadata})$ 
5:      $e_r \leftarrow \text{NEXTSTEP}(r, v_r)$ 
6:      $\mathcal{E} \leftarrow \mathcal{E} \cup \{e_r\}$ 
7:   end for
8:    $\mathcal{G} \leftarrow \text{LEGALGROUPS}(\mathcal{E}, B, w)$ 
9:   for all  $g \in \mathcal{G}$  do
10:     $(Y, \Delta S) \leftarrow \text{EXECUTEOPERATORGROUP}(g)$ 
11:     $\text{RETURNOUTPUTS}(Y, g.\text{owner\_map})$ 
12:    if  $g.\text{effect} = \text{WRITE}$  then
13:       $\text{COMMITVERSIONS}(\Delta S, g.\text{owner\_map})$ 
14:    end if
15:     $\text{UPDATEKVANDTAILMETADATA}(g)$ 
16:  end for
17: end while

```

plication, WRITE update, owner/version commit metadata, and compatibility keys. Other streaming learners fit the same runtime when they expose equivalent read/write transitions.

5 Evaluation

We organize the evaluation around five questions:

- ❶ Is the sequential serving collapse caused by mutable-state batching rather than per-request TTT cost?
- ❷ How much one-GPU serving capacity does RW-TTT recover?
- ❸ Does RW-TTT preserve the adapted checkpoint’s long-context behavior?
- ❹ Do owner or version correctness checks hold under numerical and state-contract tests?
- ❺ How broadly does the same read-write contract apply beyond the headline trace?

5.1 Experimental Setup and Evidence Scope

We evaluate a Qwen3-4B In-Place-TTT fast-weight checkpoint (Yang et al., 2025; Feng et al., 2026). Headline serving results use one GPU, BF16 cached decode, eight request-private CUDA streams, 4096 prompt tokens, 512 decode tokens, and TTT chunk size 128. We report scheduler-inclusive aggregate generated-token throughput, including grouping, waiting, fallback, and state movement.

Baselines. *Sequential In-Place TTT* (29.51 tok/s) is the original correctness-preserving serial mutable-state execution: one stream at a time. *Naive sequential replicas* place multiple unmodified batch-1 TTT processes on one GPU; at three replicas (~ 32.50 GiB) this reaches 79.78 tok/s and is the memory-matched deployment reference.

Method	Trace	Throughput	Peak	Speedup	
		tok/s	GiB	vs. serial	vs. replicas
Serial TTT	uniform	29.51	—	1.00×	—
3 serial replicas	uniform	79.78	32.50	2.70×	1.00×
Phase grouping	uniform	226.24	33.39	7.67×	2.84×
RW-TTT full	uniform	274.61	33.53	9.31×	3.44×
RW-TTT full	bursty update	177.55	33.38	6.02×	2.23×
RW-TTT full	all-update stress	174.66	33.42	5.92×	2.19×

Table 4: One-GPU serving throughput for request-owned TTT state with eight streams, 4096 prompt tokens, and 512 decode tokens. Throughput is scheduler-inclusive. The first four rows form the headline comparison against serial execution and memory-matched replicas; the final two rows stress the scheduler with higher update pressure.

Static no-TTT rows (Appendix B) provide a shared-weight capacity context but are *not* speedup denominators because they operate under a different read-only serving contract.

Evidence scope. We separate throughput measurement from behavior validation. The throughput harness measures scheduler-inclusive aggregate tok/s under controlled traces. Full task behavior is evaluated on RULER-32K/64K (Hsieh et al., 2024), because throughput measurement and long-context task execution answer different validation questions. Mutable-state correctness is further validated by numerical gates and contract stress tests (§5.5).

5.2 Batching Bottleneck Diagnosis

We first isolate single-request overhead from batching loss. The measured RW-TTT single-request overhead is small, so the sequential serving collapse is primarily a batching failure.

Controlled batch-1 timing shows that RW-TTT’s per-request execution overhead is modest: median 1.4% total, 2.1% prefill, and 0.4% decode. The serving bottleneck is therefore the inability to safely batch request-owned mutable streams. The sequential path yields only 29.51 tok/s for eight concurrent requests because it serializes all mutable-state effects.

5.3 GPU Serving Throughput

We next measure aggregate throughput under the one-GPU memory class. RW-TTT recovers most of the lost serving capacity and remains effective under higher write pressure.

Table 4 reports the main throughput and stress comparisons. The full RW-TTT path achieves 274.61 tok/s: **9.31×** over the sequential path and **3.44×** over memory-matched replicas. The uni-

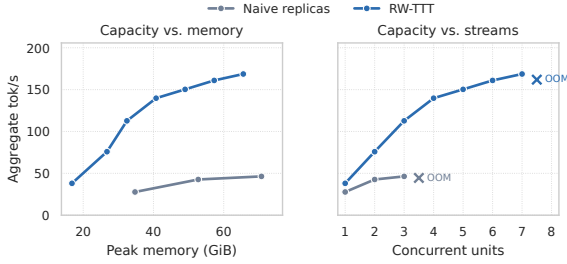


Figure 3: Scaling under a 16K prompt. Left: feasible runs ordered by peak memory. Right: same runs ordered by concurrent units. RW-TTT keeps request-private streams inside one read-write runtime.

Benchmark	In-Place TTT	RW-TTT
RULER-32K	76.30	76.41
RULER-64K	67.72	67.72

Table 5: Full RULER task scores for the adapted checkpoint executed by the original In-Place TTT path and by RW-TTT. Scores are averages over 13 tasks with 100 examples each; per-task breakdowns are in Appendix D.

form trace contains 4064 read-owner steps and 32 update-owner steps over 4096 generated tokens. Figure 3 show the scaling under the fixed memory and concurrent setting. Under bursty-update and all-update stress traces, RW-TTT remains above $5.9\times$ the sequential baseline, showing that throughput recovery is not limited to favorable read-dominant schedules. Additional denominator rows are in Appendix C.

5.4 Long-Context Behavior Preservation

We then check whether executing the adapted checkpoint through RW-TTT changes long-context task behavior. We compare against the original In-Place TTT path. Table 5 reports full RULER evaluation scores. The same adapted checkpoint is executed by the original In-Place TTT path and by RW-TTT on RULER, a 13-task long-context evaluation suite (Hsieh et al., 2024). The 32K score is essentially unchanged (76.41 vs. 76.30) and the 64K score is identical (67.72), with the full per-task breakdown retained in Appendix D.

5.5 Correctness Checks

We combine task scores, calibrated numerical gates, and explicit state-contract stress tests to check the owner/version contract. All three evidence layers support the same correctness conclusion, showing in Table 6.

These layers address complementary failure

Check	Scope	Result
Task behavior	RULER-32K/64K	preserved
NLL/PPL	9 configs $\times$ 512 tokens	9/9 pass
Relative drift	$\alpha=1.0$ gate	pass; top5_min 1.0
State-contract stress	5 injected failures	5/5 pass

Table 6: Correctness evidence summary. RULER checks task-level behavior; the NLL/PPL and relative-drift gates check calibrated numerical agreement; the state-contract row injects owner/version failure modes. Detailed RULER scores appear in Appendix D.

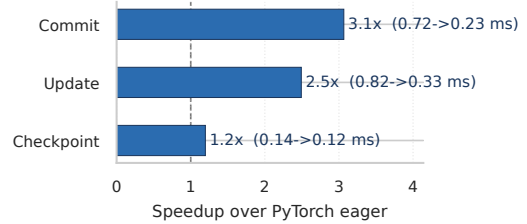


Figure 4: WRITE-side operator speedups from Triton kernels over PyTorch eager. Parentheses show latency before and after kernelization in milliseconds.

modes. RULER scores validate that the adapted model produces correct task outputs. The NLL and relative gates validate numerical fidelity of the batched execution path. State-contract stress validates owner/version/rollback invariants that average scores may not expose. Together, these checks target task preservation, calibrated numerical fidelity, and hard state invariants rather than relying on exact BF16 token identity as the sole correctness criterion.

5.6 Operator Evidence

Finally, we look beyond the headline 4K serving trace. The same contract survives breadth sweeps, exposes the 16K memory boundary, carries a second adapter-shaped mutable state, and identifies WRITE-side operator targets.

Figure 4 provides local evidence for the read-write operator surface induced by mutable TTT state. Triton kernels reduce WRITE-side latency for selective commit, dense update, and checkpoint write by $3.1\times$, $2.5\times$, and $1.2\times$, respectively. These microbenchmarks are not the end-to-end serving denominator; they show that the unusual optimization surface lies in owner-indexed commit, state update, and checkpointing rather than ordinary shared-weight projection.

6 Discussion

Phase skew remains hard. Requests are not fixed as boundary or non-boundary streams; each stream alternates between READ and WRITE steps. When updates align, the planner forms dense WRITE batches, but phase skew fragments groups. The bounded wait budget trades latency for group density without changing semantics, leaving skew-aware scheduling as future work.

State movement and layout. Read-write TTT shifts the serving surface from shared-weight projection to mutable-state commit, checkpoint, and movement. State packing and ownership tracking grow with the number of owners, making state layout and memory management the next bottleneck after batching is restored.

Operator kernels. Microbenchmarks show local wins for selective commit and dense update operators, while read-path projection does not beat `torch.bmm`. Thus WRITE-side kernelization is a targeted follow-up, not a substitute for the serving contract.

Validation scope. Our full validation uses fast-weight In-Place-TTT checkpoints, which provide a sequential reference and long-context RULER tasks. The LoRA-style experiment tests the same owner/version contract on adapter-shaped payloads, but is not a full downstream adapter-serving evaluation.

7 Related Work

KV/prefix-cache serving. Modern LLM serving has optimized the shared-weight contract through iteration-level scheduling (Yu et al., 2022), paged KV-cache management (Kwon et al., 2023), chunked-prefill/decode overlap (Agrawal et al., 2023, 2024), prefill/decode disaggregation (Zhong et al., 2024), program-level scheduling and cache reuse (Zheng et al., 2024), and further refinements in high-throughput generation, co-serving, and elastic disaggregation (Holmes et al., 2024; Qiao et al., 2024; Ruan et al., 2025; Borui et al., 2025). These systems manage dynamic *activation*-side state while model-side parameters remain shared and read-only. RW-TTT keeps the scheduling problem but changes the object being scheduled: each request also owns model-side state whose version may change during generation.

Adapter serving. S-LoRA and Punica make heterogeneous request-specific model-side state a first-class serving concern (Hu et al., 2022; Sheng et al., 2023; Chen et al., 2024; Mangrulkar et al., 2022). Their core contract is *read-side heterogeneity*: different requests apply different low-rank weights, and the serving stack manages placement and batched execution, but adapters remain read-only during generation. Read-write TTT builds on the same insight that small request-specific model state can be served efficiently, then adds online mutation: a LoRA-style TTT state can be read like an adapter, dirtied by an update event, committed under a new version, and read again by the same request. Our LoRA-style stress test confirms that the same serving interface carries adapter-shaped payloads.

Test-time training. Test-time training adapts models using information available at inference time (Sun et al., 2020). Recent work uses test-time learning inside recurrent-style hidden states (Sun et al., 2024), while In-Place-TTT applies online updates to selected LLM layers during long-context generation (Feng et al., 2026). These are back-end choices for producing mutable state and update rules. RW-TTT targets the missing serving-system layer needed when such methods read and write request-owned state during generation.

8 Conclusion

Test-time training introduces request-owned mutable model state that breaks the static-weight batching contract behind current LLM serving systems. RW-TTT restores batching through a read-write serving contract: each decode step is tagged with its owner and READ/WRITE effect, only compatible phases are grouped, and updated state versions are committed exclusively to owning requests. On eight concurrent streams with a fast-weight In-Place-TTT checkpoint, this recovers 274.61 aggregate tok/s— $9.31\times$ over the sequential TTT path and $3.44\times$ over memory-matched independent replicas—while preserving adapted RULER-32K/64K behavior and passing owner/version correctness gates. The contribution is a serving contract and batching runtime for request-owned mutable state, not a new TTT learning rule or kernel system; RW-TTT separates TTT backend design from the serving semantics needed to batch online-adaptive model state.

Limitations

RW-TTT evaluates the serving layer for existing online TTT behavior rather than proposing a new adaptation rule. We therefore hold the backend update rule fixed and ask whether request-owned mutable state can be batched without changing the adapted checkpoint’s behavior. Full task evaluation uses the fast-weight In-Place-TTT checkpoints because they provide both a sequential reference and long-context adapted checkpoints; the LoRA-style experiment exercises the same owner/version READ/WRITE contract on adapter-shaped mutable state. The throughput ladder is reported for the one-GPU memory class and eight-stream workload used by the prototype, with comparisons against the original sequential TTT path and memory-matched unmodified TTT replicas. Static-weight serving is a different contract: RW-TTT targets the case where model-side request state is read and written during generation.

References

- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, pages 117–134.
- Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*.
- Wan Borui, Zhao Juntao, Jiang Chenyu, Guo Chuanxiong, and Wu Chuan. 2025. Efficient llm serving on hybrid real-time and best-effort requests. *arXiv preprint arXiv:2504.09590*.
- Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2024. Punica: Multi-tenant lora serving. *Proceedings of Machine Learning and Systems*, 6:1–13.
- Guhaog Feng, Shengjie Luo, Kai Hua, Ge Zhang, Di He, Wenhao Huang, and Tianle Cai. 2026. In-place test-time training. *arXiv preprint arXiv:2604.06169*.
- Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and 1 others. 2024. DeepSpeed-fastgen: High-throughput text generation for llms via mii and deepSpeed-inference. *arXiv preprint arXiv:2401.08671*.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. Ruler: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, and 1 others. 2022. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3.
- Jinwu Hu, Zhitian Zhang, Guohao Chen, Xutao Wen, Chao Shuai, Wei Luo, Bin Xiao, Yuanqing Li, and Minghui Tan. 2025. Test-time learning for large language models. *arXiv preprint arXiv:2505.20633*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.
- Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. 2022. Peft: State-of-the-art parameter-efficient fine-tuning methods.
- Yifan Qiao, Shu Anzai, Shan Yu, Haoran Ma, Shuo Yang, Yang Wang, Miryung Kim, Yongji Wu, Yang Zhou, Jiarong Xing, and 1 others. 2024. Conserve: Fine-grained gpu harvesting for llm online and offline co-serving. *arXiv preprint arXiv:2410.01228*.
- Chaoyi Ruan, Yinhe Chen, Dongqi Tian, Yandong Shi, Yongji Wu, Jialin Li, and Cheng Li. 2025. Dynaserve: Unified and elastic execution for dynamic disaggregated llm serving. *arXiv preprint arXiv:2504.09285*.
- Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, and 1 others. 2023. S-lora: Serving thousands of concurrent lora adapters. *arXiv preprint arXiv:2311.03285*.
- Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, and 1 others. 2024. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*.
- Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. 2020. Test-time training with self-supervision for generalization under distribution shifts. In *International conference on machine learning*, pages 9229–9248. PMLR.
- Arnub Tandon, Karan Dalal, Xinhao Li, Daniel Kocaja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, and 1 others. 2025. End-to-end test-time training for long context. *arXiv preprint arXiv:2512.23675*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI 22)*, pages 521–538.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, and 1 others. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210.

A Reproducibility Notes

We distinguish three throughput scopes. *Decode-only* counts only the decode execution window after prompt state is available. *Prefill+decode* also includes prompt processing. *Scheduler-inclusive* further includes grouping, waiting, fallback, and state movement. The main-text serving rows use one canonical scheduler-inclusive scope; other scopes appear only in the appendix denominator map.

Item	Setting
Primary model	Qwen3-4B In-Place-TTT checkpoint
Precision	BF16 cached decode; FP32 spot checks
Prompt/decode	4096 prompt tokens, 512 decode tokens
TTT chunk	128 for prototype serving harness
Target streams	8 request-private streams
Throughput scope	scheduler-inclusive unless stated otherwise
Memory	one-GPU memory class around 33 GiB for the main rows

Table 7: Compact setup for the main full-model serving rows.

B Denominator Map

Row	GPUs	Seq.	Batch	Scope tok/s	Role
Static short-context ceiling	1	1024	8	300.06	context only
Static target-context row	1	4096	1	37.08	feasible static row
Static target-context batch4	1	4096	4	OOM	prefill limit
Static target-context batch8	1	4096	8	OOM	prefill limit
In-Place TTT sequential	1	4096	8 request streams	29.51	serial denominator
Naive sequential replicas	1	4096	3 replicas	79.78	memory-matched deployment baseline
READ/WRITE phase grouping	1	4096	8 request streams	226.24	core batching component
RW-TTT full read/write	1	4096	8 request streams	274.61	full serving row
Naive independent TTT	8	4096	8 streams	252.00	topology context

Table 8: Denominator map. Rows with different GPU counts, sequence lengths, or timing scopes are context rows, not direct speedup denominators. The main-text headline comparison uses only the one-GPU, 4096-token, scheduler-inclusive rows.

C Full Serving Rows

Method	Unit count	Grouping mode	Agg. tok/s	Speedup	Role
Serial TTT	8 request streams	serial groups	29.51	1.00	denominator
3 serial replicas	3 replicas	independent batch-1 processes	79.78	2.70	memory-matched baseline
RW-TTT phase grouping	8 request streams	phase-compatible groups	226.24	7.67	core batching component
RW-TTT + owner commit	8 request streams	phase-compatible groups	226.25	7.67	commit ablation
RW-TTT full read/write	8 request streams	read/write groups	274.61	9.31	full system
RW-TTT bursty update	8 request streams	phase-skewed groups	177.55	6.02	stress setting
RW-TTT all-update stress	8 request streams	dense write groups	174.66	5.92	stress setting

Table 9: Full-model prototype serving-harness results. Speedup is relative to the serial In-Place TTT path on one GPU.

Table 9 serving harness shows that phase-compatible batching is the dominant source of throughput recovery: RW-TTT phase grouping improves aggregate throughput from 29.51 to 226.24 tok/s, while

adding owner-commit semantics does not introduce measurable overhead. The full read/write system reaches 274.61 tok/s, or $9.31\times$ over serial execution, and remains above $5.9\times$ speedup even under bursty-update and all-update stress settings.

Replicas	Status	Decode-only tok/s	Worker-elapsed tok/s	Startup-incl. tok/s	Peak GiB
1	ok	38.76	21.42	6.31	10.84
2	ok	73.76	42.03	33.08	21.67
3	ok	79.78	44.91	36.90	32.50
4	ok	86.62	48.87	41.66	43.33

Table 10: Single-GPU naive deployment baseline using multiple original sequential In-Place TTT batch-1 processes on one GPU. Three replicas are the memory-matched baseline for the main RW-TTT comparison; four replicas exceed that memory budget.

Table 10 naive replica baseline improves throughput by placing more independent batch-1 processes on the same GPU, but the gains quickly saturate as memory usage grows. The memory-matched three-replica point reaches 79.78 decode-only tok/s at 32.50 GiB, which is substantially below the 274.61 tok/s achieved by the full RW-TTT read/write system under a comparable memory budget.

D RULER Per-Task Scores

Task	In-Place TTT	Repeat	RW-TTT
ruler_qa_squad_32k	45.00	45.00	46.00
ruler_qa_hotpotqa_32k	48.00	48.00	48.00
ruler_niah_single_1_32k	100.00	100.00	100.00
ruler_niah_single_2_32k	89.00	89.00	90.00
ruler_niah_single_3_32k	99.00	99.00	99.00
ruler_niah_multikey_1_32k	66.00	66.00	66.00
ruler_niah_multikey_2_32k	82.00	82.00	82.00
ruler_niah_multikey_3_32k	32.00	32.00	32.00
ruler_niah_multivalue_32k	97.75	97.75	97.75
ruler_niah_multiquery_32k	94.50	94.50	94.50
ruler_vt_32k	99.80	99.80	99.60
ruler_fwe_32k	96.00	96.00	95.67
ruler_cwe_32k	42.80	42.80	42.80

Table 11: Full step-aligned 32K RULER task scores for the adapted fast-weight checkpoint.

E Breadth and 16K Capacity Stress

Breadth . 36 trace/configuration rows tested across varying boundary fractions, TTT layer selections, and sequence lengths. All 36 rows complete successfully with no OOM events and no worker failures.

16K capacity stress Under 16384 prompt tokens and 512 decode tokens with 7 concurrent streams:

F PEFT/LoRA Adapter-State Path Check

The LoRA-style experiment maps rank-8 PEFT adapter states to `DeltaAdapterState` and exercises the same ownership, versioning, READ/WRITE, and commit path used by fast-weight TTT state. This is an interface and path check, not a downstream LoRA-TTT quality benchmark.

Four materialized adapter/domain pairs are tested. All pass exact apply/update operator equivalence checks with adapter-file NLL changes below 3.4×10^{-3} , confirming that the mutable-state serving contract carries a second payload shape without corruption.

Task	In-Place TTT	RW-TTT	Delta
ruler_qa_squad_64k	44.00	44.00	0.00
ruler_qa_hotpotqa_64k	44.00	44.00	0.00
ruler_niah_single_1_64k	100.00	100.00	0.00
ruler_niah_single_2_64k	59.00	59.00	0.00
ruler_niah_single_3_64k	96.00	96.00	0.00
ruler_niah_multikey_1_64k	34.00	34.00	0.00
ruler_niah_multikey_2_64k	85.00	85.00	0.00
ruler_niah_multikey_3_64k	22.00	22.00	0.00
ruler_niah_multivalue_64k	95.25	95.25	0.00
ruler_niah_multiquery_64k	94.75	94.75	0.00
ruler_vt_64k	100.00	100.00	0.00
ruler_fwe_64k	94.00	94.00	0.00
ruler_cwe_64k	12.40	12.40	0.00

Table 12: Full 64K RULER task scores for the matching qwen3-4b-64k-ttt-step1250 checkpoint.

Method	Streams	Agg. tok/s	Peak GiB
In-Place TTT sequential	7	36.13	44.39
READ/WRITE phase grouping	7	152.58	65.58
RW-TTT full	7	168.66	65.58

Table 13: 16K capacity stress results. These are stress-test points, not the headline 4096-token denominator. At 8 streams the configuration exceeds GPU memory, scoping the memory boundary for longer sequences.

G Operator Microbench Details

The read-write serving contract fixes the operator boundary: READ steps consume a versioned state view; WRITE steps compute updated state and commit it. On representative H800 BF16 shapes (Qwen3-4B dimensions), Triton kernels show WRITE-side opportunities:

Operator	Best speedup	Median speedup	Win rows / total	Scope
Selective commit (ActiveStateUpdateInplace)	3.07×	2.26×	90/90	dirty-only write
Fused update + writeback (StateUpdateFused)	2.49×	2.04×	24/24	dense write
Checkpoint write	1.20×	1.19×	9/9	rollback snapshot

Table 14: WRITE-side operator opportunities. Read-path projection (MutableWeightLinear, FusedReadResidual) does not beat torch.bmm/cuBLAS in the current prototype. These are local microbenchmark wins, not integrated Triton serving speedups.

H Serving Loop Pseudocode

Algorithm 1 in the main text gives the compact serving iteration. The full loop below adds queue management, wait-budget logic, and fallback handling.

Algorithm 2 RW-TTT TTT-aware serving loop (full)

Require: active requests $\mathcal{R}$, target batch size B , wait budget w

```
1: while  $\mathcal{R}$  is not empty do
2:    $\mathcal{E} \leftarrow \emptyset$ 
3:   for all  $r \in \mathcal{R}$  do
4:      $v_r \leftarrow \text{VIEW}(r.\text{TTTState}, r.\text{metadata})$ 
5:      $e_r \leftarrow \text{NEXTSTEP}(r, v_r)$   $\triangleright$  READ or WRITE
6:      $\mathcal{E} \leftarrow \mathcal{E} \cup \{e_r\}$ 
7:   end for
8:    $\mathcal{G} \leftarrow \text{PLANTTTAWAREBATCHES}(\mathcal{E}, B, w)$ 
9:   for all  $g \in \mathcal{G}$  do
10:     $(Y, \Delta S) \leftarrow \text{EXECUTEOPERATORGROUP}(g)$ 
11:     $\text{RETURNOUTPUTS}(Y, g.\text{owner\_map})$ 
12:    if  $g.\text{effect} = \text{WRITE}$  then
13:       $\text{COMMITVERSIONS}(\Delta S, g.\text{owner\_map})$ 
14:    end if
15:     $\text{UPDATEKVANDTAILMETADATA}(g)$ 
16:  end for
17: end while
```

I Mechanism-Level Comparison

System class	Model-side state	Mutates	Update-aware batch	Contract
Static LLM serving	no	no	no	shared weights + KV cache
Paged KV / continuous batching	no	no	no	activation-side cache placement
Adapter serving	selected adapter	usually no	read-side adapter batch	read-only low-rank weights
Serial TTT execution	yes	yes	no	one mutable stream at a time
RW-TTT	yes	yes	yes	owner-indexed read/write state

Table 15: Mechanism-level comparison with existing serving abstractions.